LABORATORY OBSERVATION / RECORD

Course Full Stack Web Development
Course Code 23CM4121
Experiment No. 5
Student Name K. Anusha
Roll No. A24126552088
Date 28-10-2026

1. TITLE

Differentiate between JavaScript Functions and Classes

2. AIM

To differentiate between JavaScript functions and classes, and to show how a class can be used to create multiple objects with common properties and methods.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how a constructor function creates objects using the new keyword.
- To add shared methods to a constructor function using its prototype.
- To create a class using the class keyword with a constructor() and methods.
- To create multiple objects from the same class that share common properties and methods.
- To compare the function-based approach with the class-based approach in JavaScript.

4. THEORY

Function (constructor function): In JavaScript, a normal function can act as a constructor when it is called with the new keyword. The properties are assigned using this, and the methods are added to the function's prototype so that they are shared by all objects. In this experiment, StudentFunction is a constructor function.

Class: A class is an ES6 feature that provides a cleaner syntax for creating objects. It contains a constructor() method to initialise the properties and other methods that are written inside the class body. In this experiment, Student is a class.

Object: An object is an instance created from a function or a class. Many objects can be created from the same class, and each object has its own property values but uses the same methods.

Difference: A function can be called with or without new, and its methods are attached manually through the prototype. A class must always be called with new, and its properties and methods are written together in one block, which makes the code easier to read and organise.

5. ALGORITHM / PROCEDURE

- Create a constructor function StudentFunction(name, course) and assign the properties using this.

- Add the method showInfo() to StudentFunction.prototype.
- Create an object s1 using the new keyword and call s1.showInfo().
- Create a class Student with a constructor(name, course) that assigns the properties.
- Add the method showInfo() inside the Student class.
- Create multiple objects s2 and s3 from the Student class using the new keyword.
- Call showInfo() on each object and observe the output.
- Run the program using node and verify the output in the terminal.

6. PROGRAM

functions_classes.js

```
function StudentFunction(name, course) {
    this.name = name;
    this.course = course;
}
// Methods added via prototype (shared across all instances)
StudentFunction.prototype.showInfo = function () {
    console.log(`${this.name} is enrolled in ${this.course}`);
};

const s1 = new StudentFunction("Aarav", "CSE");
s1.showInfo(); // Aarav is enrolled in CSE


// --- Using a class (ES6) ---
class Student {
    constructor(name, course) {
        this.name = name;      // property
        this.course = course;  // property
    }

    showInfo() {                // method
        console.log(`${this.name} is enrolled in ${this.course}`);
    }
}

// Creating multiple objects from the same class
const s2 = new Student("Priya", "IT");
const s3 = new Student("Rahul", "BCA");

s2.showInfo(); // Priya is enrolled in IT
s3.showInfo();
```

7. CODE EXPLANATION

Code	Explanation
function StudentFunction(name, course)	Defines a constructor function that receives the name and course values.
this.name = name;	Stores the name in a property of the object being created.
StudentFunction.prototype.showInfo	Adds a method to the prototype so that it is shared by all objects of the function.
new StudentFunction("Aarav", "CSE")	Creates the object s1 by calling the constructor function with new.

Code	Explanation
class Student	Defines a class using the ES6 class syntax.
constructor(name, course)	Special method of a class that runs when an object is created and sets the properties.
showInfo()	Method written inside the class that prints the student details.
new Student("Priya", "IT")	Creates the object s2 from the Student class.
new Student("Rahul", "BCA")	Creates the object s3 from the same class with different values.
console.log(`...`)	Prints the message to the terminal using a template literal.

8. TEST CASES AND EXECUTION DATA

Each part of the program was executed and the actual output was recorded against the expected output.

Test Case	Component	Action	Expected Result	Actual Result	Status
TC-01	Project	Run node functions_classes.js	Program runs without errors	Program executed successfully	Pass
TC-02	StudentFunction	Call s1.showInfo()	Aarav is enrolled in CSE	Aarav is enrolled in CSE	Pass
TC-03	Student class	Call s2.showInfo()	Priya is enrolled in IT	Priya is enrolled in IT	Pass
TC-04	Student class	Call s3.showInfo()	Rahul is enrolled in BCA	Rahul is enrolled in BCA	Pass
TC-05	Student class	Check that s2 and s3 have different values	Each object holds its own name and course	Each object displayed its own details	Pass

9. OUTPUT

The program was run using Node.js and the output was checked in the terminal.

Output 1 (TC-01 to TC-05): Terminal output

```
PS C:\Users\HP\FS lab> node functions_classes.js
Aarav is enrolled in CSE
Priya is enrolled in IT
Rahul is enrolled in BCA
```

10. RESULT

Thus, the difference between JavaScript functions and classes was understood. A constructor function and a class were created, and multiple objects were created from the Student class sharing the same properties and methods. All test cases were executed and passed.

LABORATORY OBSERVATION / RECORD

Course Full Stack Web Development
Course Code 23CM4121
Experiment No. 6
Student Name K. Anusha
Roll No. A24126552088
Date 28-10-2026

1. TITLE

Node.js Modules: os, path and fs

2. AIM

To understand Node.js modules and to demonstrate the purpose of the built-in os, path and fs modules with suitable examples.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what a module is and why modularity is important in Node.js.
- To use the os module to get information about the operating system.
- To use the path module to work with file and directory paths.
- To use the fs module to create and write a file.
- To import built-in modules using require().
- To run the program and verify the output in the terminal.

4. THEORY

Node.js module: A module is a separate file or package that contains related code, such as functions and objects, which can be imported into other files using require(). Modules keep the code organised, reusable and easy to maintain. This concept is called modularity.

Types of modules: Node.js has built-in (core) modules such as os, path and fs, which need no installation; local modules, which are created by the programmer; and third-party modules, which are installed using NPM.

os module: It provides information about the operating system, such as the platform, CPU architecture, total memory, free memory and home directory.

path module: It is used to work with file and directory paths. It can find the file name, directory name and extension of a path and can join path segments in the correct format for the operating system.

fs module: The fs (file system) module is used to read, write, update and delete files. The writeFile() method writes data to a file asynchronously and creates the file if it does not exist.

5. ALGORITHM / PROCEDURE

- Create a file named modules.js in the project folder.
- Import the os module using require('os') and display the platform, CPU architecture, total memory, free memory and home directory.
- Import the path module using require('path').
- Use path.basename(), path.dirname(), path.extname() and path.join() on a sample file path.
- Import the fs module using require('fs').
- Use fs.writeFile() to create the file note.txt with the text "Hello from fs module!".
- Run the program using node modules.js.
- Verify the output in the terminal and check that note.txt is created.

6. PROGRAM

modules.js

```
const os = require('os');

console.log('Platform:', os.platform());       // e.g. 'linux', 'win32'
console.log('CPU Architecture:', os.arch());    // e.g. 'x64'
console.log('Total Memory (MB):', os.totalmem() / (1024 * 1024));
console.log('Free Memory (MB):', os.freemem() / (1024 * 1024));
console.log('Home Directory:', os.homedir());



// ---- path module: working with file/directory paths ----
const path = require('path');

const filePath = '/home/user/documents/report.txt';
console.log('File name:', path.basename(filePath));   // report.txt
console.log('Directory:', path.dirname(filePath));     // /home/user/documents
console.log('Extension:', path.extname(filePath));     // .txt
console.log('Joined path:', path.join('folder', 'subfolder', 'file.js'));



// ---- fs module: file handling ----
const fs = require('fs');

fs.writeFile('note.txt', 'Hello from fs module!', (err) => {
    if (err) throw err;
    console.log('File written successfully.');
});
```

7. CODE EXPLANATION

Code	Explanation
const os = require('os');	Imports the built-in os module.
os.platform()	Returns the operating system platform, such as win32 or linux.
os.arch()	Returns the CPU architecture, such as x64.
os.totalmem() / (1024 * 1024)	Gives the total system memory converted from bytes to MB.
os.freemem() / (1024 * 1024)	Gives the free system memory converted from bytes to MB.

Code	Explanation
os.homedir()	Returns the home directory of the current user.
const path = require('path');	Imports the built-in path module.
path.basename(filePath)	Returns the last part of the path, which is the file name.
path.dirname(filePath)	Returns the directory part of the path.
path.extname(filePath)	Returns the file extension.
path.join(...)	Joins the path segments using the separator of the operating system.
const fs = require('fs');	Imports the built-in file system module.
fs.writeFile(...)	Writes text to note.txt asynchronously and creates the file if it does not exist.
if (err) throw err;	Stops the program and shows the error if the file could not be written.

8. TEST CASES AND EXECUTION DATA

Each part of the program was executed and the actual output was recorded against the expected output.

Test Case	Component	Action	Expected Result	Actual Result	Status
TC-01	Project	Run node modules.js	Program runs without errors	Program executed successfully	Pass
TC-02	os module	Display platform, architecture and memory	System details are printed	Platform, architecture, memory and home directory displayed	Pass
TC-03	path module	Use basename, dirname and extname	report.txt, /home/user/documents and .txt	report.txt, /home/user/documents and .txt	Pass
TC-04	path module	Use path.join()	Path joined with the OS separator	folder\subfolder\file.js	Pass
TC-05	fs module	Write note.txt using fs.writeFile()	File created and message shown	File written successfully. and note.txt created	Pass

9. OUTPUT

The program was run using Node.js and the output was checked in the terminal. The values of the os module depend on the system on which the program is run.

Output 1 (TC-01 to TC-05): Terminal output

```
PS C:\Users\HP\FS lab> node modules.js
Platform: win32
CPU Architecture: x64
Total Memory (MB): 16221.35
Free Memory (MB): 6347.82
Home Directory: C:\Users\HP
File name: report.txt
Directory: /home/user/documents
Extension: .txt
Joined path: folder\subfolder\file.js
```

```
File written successfully.
```

Output 2 (TC-05): File created

The file note.txt was created in the project folder with the following content.

```
Hello from fs module!
```

10. RESULT

Thus, the Node.js modules os, path and fs were studied and used successfully. System information was displayed using os, path details were obtained using path, and a file was created using fs. All test cases were executed and passed.

LABORATORY OBSERVATION / RECORD

Course Full Stack Web Development
Course Code 23CM4121
Experiment No. 7
Student Name K. Anusha
Roll No. A24126552088
Date 28-10-2026

1. TITLE

NPM and package.json in Node.js

2. AIM

To understand NPM and package.json in Node.js, and to install an external package (Express) and use it in a Node.js application.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what NPM (Node Package Manager) is and why it is used.
- To understand the purpose of the package.json file and its dependencies.
- To create a package.json file using npm init.
- To install an external package using npm install.
- To use the installed package in a Node.js application using require().
- To run the application and verify the output in the browser.

4. THEORY

NPM: NPM (Node Package Manager) is the default package manager of Node.js. It is used to install, update and remove external packages (libraries) that are published on the NPM registry.

package.json: It is a JSON file in the root of a Node.js project. It stores the project name, version, entry file (main), scripts and the list of dependencies with their versions. It helps others to install the same packages using npm install.

Dependencies: These are the external packages required by the project. When a package is installed with npm install, it is saved in the dependencies section of package.json and downloaded into the node_modules folder.

Express: Express is a popular third-party web framework for Node.js. It is used to create web servers and define routes easily. In this experiment, Express is installed using NPM and used to create a simple server.

Common NPM commands: npm init -y creates package.json, npm install <package> installs a package, npm uninstall <package> removes it and npm list shows the installed packages.

5. ALGORITHM / PROCEDURE

- Create a new project folder and open it in the terminal.
- Run npm init -y to create the package.json file.
- Run npm install express to install the Express package.
- Check that Express is added to the dependencies section of package.json and that the node_modules folder is created.
- Create the file express.js and import Express using require('express').
- Create the app using express() and define a route for "/" that sends a message.
- Start the server using app.listen() on port 3000.
- Run the application using node express.js and open http://localhost:3000 in the browser.

6. PROGRAM

package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "main": "index.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

express.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
    res.send('Express package is working!');
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

7. CODE EXPLANATION

Code	Explanation
"name": "my-app"	Name of the project.
"version": "1.0.0"	Version number of the project.
"main": "index.js"	Entry point file of the project.
"dependencies"	List of external packages required by the project.
"express": "^4.18.2"	Express package with version 4.18.2 or a later compatible 4.x version.
require('express')	Imports the installed Express package.
const app = express();	Creates an Express application object.
app.get('/', (req, res) => {...})	Defines a route that handles GET requests to the home page.
res.send(...)	Sends the text response to the browser.
app.listen(3000, ...)	Starts the server on port 3000 and prints a message when it is ready.

8. TEST CASES AND EXECUTION DATA

Each part of the application was executed and the actual output was recorded against the expected output.

Test Case	Component	Action	Expected Result	Actual Result	Status
TC-01	NPM	Run npm install express	Express is installed	Package installed without errors	Pass
TC-02	package.json	Open package.json	express added under dependencies	"express": "^4.18.2" is present	Pass
TC-03	Server	Run node express.js	Server starts on port 3000	Server running on port 3000	Pass
TC-04	Route /	Open http://localhost:3000	Message is displayed in the browser	Express package is working!	Pass

9. OUTPUT

The commands were run in the terminal and the output was checked in the browser. The output for each test case is given below.

Output 1 (TC-01, TC-02): Installing the package

```
PS C:\Users\HP\FS lab\my-app> npm install express
added 65 packages, and audited 66 packages in 4s
found 0 vulnerabilities
```

Output 2 (TC-03): Server start

```
PS C:\Users\HP\FS lab\my-app> node express.js
Server running on port 3000
```

Output 3 (TC-04): Browser output

The browser shows the following message at http://localhost:3000. (Paste your browser screenshot here.)

```
Express package is working!
```

10. RESULT

Thus, NPM and package.json were studied. The Express package was installed using NPM, added to the dependencies of package.json, and used to create a simple server in a Node.js application. All test cases were executed and passed.

LABORATORY OBSERVATION / RECORD

Course Full Stack Web Development
Course Code 23CM4121
Experiment No. 8
Student Name K. Anusha
Roll No. A24126552088
Date 28-10-2026

1. TITLE

Middleware in Express.js

2. AIM

To understand middleware in Express.js, to explain how it works in the request-response cycle, and to develop a middleware that logs incoming requests.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what middleware is in Express.js.
- To understand how middleware works in the request-response cycle.
- To create a custom middleware function with req, res and next.
- To apply the middleware to all routes using app.use().
- To log the date, time, method and URL of every incoming request.
- To run the application and verify the logs in the terminal.

4. THEORY

Middleware: Middleware is a function that runs between the request coming from the client and the response sent by the server. It has access to the request object (req), the response object (res) and the next() function.

Request-response cycle: When a request arrives, Express passes it through the middleware functions in the order in which they are defined. Each middleware can read or change req and res, end the response, or call next() to pass control to the next middleware or route handler.

next(): If a middleware does not end the response and does not call next(), the request will stay pending and the browser will keep waiting. Therefore next() must be called to continue the cycle.

app.use(): It registers a middleware that is applied to all incoming requests. In this experiment, requestLogger is a custom middleware that prints the time, method and URL of each request, and then calls next().

Uses of middleware: Logging, authentication, parsing request data, error handling and serving static files.

5. ALGORITHM / PROCEDURE

- Create a project folder, run npm init -y and install Express using npm install express.
- Create the file middleware.js and import Express.
- Define the function requestLogger(req, res, next) that creates a timestamp and prints the method and URL.
- Call next() at the end of the middleware to pass control forward.
- Apply the middleware to all requests using app.use(requestLogger).
- Create the routes "/" and "/students" that send a response.
- Start the server on port 3000 using app.listen().
- Run node middleware.js, open both routes in the browser and check the logs in the terminal.

6. PROGRAM

middleware.js

```
const express = require('express');
const app = express();

// ---- Custom logging middleware ----
function requestLogger(req, res, next) {
    const timestamp = new Date().toISOString();
    console.log(`[${timestamp}] ${req.method} request to ${req.url}`);
    next(); // pass control to the next middleware / route handler
}

// Apply middleware to ALL incoming requests
app.use(requestLogger);

app.get('/', (req, res) => {
    res.send('Home Page');
});

app.get('/students', (req, res) => {
    res.send('Students List');
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

7. CODE EXPLANATION

Code	Explanation
const express = require('express');	Imports the Express package.
function requestLogger(req, res, next)	Defines the middleware function with the request, response and next parameters.
new Date().toISOString()	Creates the current date and time as a text timestamp.
console.log(`[...] ${req.method} request to ${req.url}`)	Prints the timestamp, the HTTP method and the URL of the request.
next();	Passes control to the next middleware or route handler.
app.use(requestLogger);	Applies the middleware to all incoming requests.

Code	Explanation
app.get('/', ...)	Route that sends "Home Page" as the response.
app.get('/students', ...)	Route that sends "Students List" as the response.
app.listen(3000, ...)	Starts the server on port 3000.

8. TEST CASES AND EXECUTION DATA

Each part of the application was executed and the actual output was recorded against the expected output.

Test Case	Component	Action	Expected Result	Actual Result	Status
TC-01	Server	Run node middleware.js	Server starts on port 3000	Server running on port 3000	Pass
TC-02	Route /	Open http://localhost:3000/	Home Page is displayed and the request is logged	Home Page displayed; GET request to / logged	Pass
TC-03	Route /students	Open http://localhost:3000/students	Students List is displayed and the request is logged	Students List displayed; GET request to /students logged	Pass
TC-04	Middleware	Check the log format	Timestamp, method and URL are printed	[timestamp] GET request to /url	Pass
TC-05	next()	Check that routes respond after logging	Response is received without delay	Both pages loaded normally	Pass

9. OUTPUT

The application was run using Node.js and the output was checked in the browser and in the terminal. The timestamp changes each time a request is made.

Output 1 (TC-01 to TC-04): Terminal output

```
PS C:\Users\HP\FS lab\my-app> node middleware.js
Server running on port 3000
[2026-10-28T10:15:32.481Z] GET request to /
[2026-10-28T10:15:41.209Z] GET request to /students
```

Output 2 (TC-02, TC-03, TC-05): Browser output

The browser shows the following pages. (Paste your browser screenshots here.)

```
http://localhost:3000/          ->  Home Page
http://localhost:3000/students  ->  Students List
```

10. RESULT

Thus, the concept of middleware in Express.js was understood. A custom logging middleware was created and applied to all requests using app.use(), and every incoming request was logged with its timestamp, method and URL. All test cases were executed and passed.